

Trabalho Prático I: O Problema da Ponte e da Tocha

Fundamentos de Inteligência Artificial

Autores:

Davi Paulo Vilela

Gustavo Campana Do Vale

Otávio Augusto Bernardes Rodrigues

Vinícius Moreira Faria

21 de setembro de 2026

Resumo

Este relatório apresenta a modelagem de um quebra-cabeça como um problema de busca em espaço de estados, bem como a análise e resultados obtidos ao comparar quatro agentes executando algoritmos de Busca informada e desinformada para resolver o problema. Os experimentos realizados cobriram diferentes instâncias do problema para melhor análise dos algoritmos implementados. Os resultados demonstram a tese central de que a busca informada com heurística admissível reduz a quantidade de nós buscados e o tempo de processamento sem perder a garantia de otimalidade.

1 Introdução

A Busca em espaço de estados é uma técnica que permite resolver problemas de busca explorando um grafo cujos vértices são os estados possíveis e cujas arestas são as ações que levam de um estado para outro. Neste trabalho, exploramos a diferença entre a busca não informada e a busca baseada na informação sobre a distância ao objetivo.

O problema modelado é o quebra-cabeça conhecido como "Problema da Ponte e da Tocha": n pessoas precisam atravessar uma ponte à noite, no máximo duas por vez, sempre carregando uma única tocha. A pessoa i atravessa sozinha em c_i minutos; duas pessoas juntas avançam na velocidade da mais lenta.

2 Modelagem do problema

2.1 Espaço de estados

Um estado representa todas as informações necessárias para definir qualquer outro estado relacionado. Nesse caso, a configuração completa de quem já atravessou a ponte e onde está a tocha. No código, o estado foi representado como uma máscara de bits de $n+1$ posições:

$$s = (b_0, b_1, \dots, b_{n-1}, t), \quad b_i = 1 \Leftrightarrow \text{pessoa } i \text{ já cruzou}, \quad t = 1 \Leftrightarrow \text{tocha do lado oposto}.$$

A escolha por bitmask em detrimento de um vetor de booleanos facilitou a manipulação limpa do estado. O conjunto de explorados é um `unordered_set<uint32_t>` com custo $O(1)$ por consulta. O número de estados possíveis, ou o Espaço de Estados completo é dado por $|S| = 2^{n+1}$.

2.2 Estado inicial e objetivo

$$s_0 = \underbrace{00 \dots 0}_{n+1} \quad (\text{Todo mundo no início da ponte}) \quad G = \underbrace{11 \dots 1}_{n+1} \quad (\text{Todo mundo no fim da ponte}).$$

O teste de objetivo é uma comparação de igualdade de máscaras ($s = G$), executada na *expansão* de cada nó.

2.3 Função sucessora, ações e custos

De um estado s , as ações válidas dependem do lado da tocha, somente podendo atravessar quem está do mesmo lado da tocha, em grupos de uma ou duas pessoas. A função sucessora enumera todas as ações válidas:

- **Travessias de 1 pessoa:** para cada pessoa i do lado da tocha, retorna s' com b_i e t invertidos; custo $c(s, s') = c_i$.
- **Travessias de 2 pessoas:** para cada par $\{i, j\}$ do lado da tocha, retorna s' com b_i , b_j e t invertidos. Custo $c(s, s') = \max(c_i, c_j)$.

O grafo resultante é direcionado, uma vez que somente quem está do lado da tocha pode atravessar, e ponderado pelo tempo de travessia.

2.4 Função de custo

$$g(s) = \sum_k c(s_k, s_{k+1}) = \text{tempo total acumulado da raiz até } s.$$

No código, g é armazenado no nó e somado de forma incremental, sem recálculo.

3 Métodos de Busca em Espaço de Estados

3.1 Rotinas de implementação

Os quatro métodos foram implementados em um único laço de busca em grafo que diferencia os algoritmos escolhidos apenas alterando a política de prioridade da fronteira. Dessa forma, a análise elimina pequenas diferenças entre implementações e garante que todos os métodos fazem o teste de objetivo no mesmo ponto, usam o mesmo conjunto de explorados e a mesma contagem de expansões. A comparação mede apenas a estratégia de cada algoritmo, não o código. Além disso, outros três cuidados foram essenciais:

1. **Lista de nós expandidos:** Entradas de um mesmo estado alcançado por caminho pior são descartadas sem contar como expansão. Sem isso a Busca de Menor Custo perderia a otimalidade quando um caminho melhor chega depois de um pior.
2. **Lista de nós:** os nós vivem em um **vector** e apontam para o pai por índice, eliminando alocação de instâncias individuais e nos caminhos críticos para a análise.
3. **Limite de segurança:** teto de expansões configurável, conforme recomenda o enunciado. Em nenhum experimento o limite (15000) foi atingido.

3.2 Métodos de busca desinformada

A **Busca em Profundidade (DFS)** é um método de busca que expande o nó mais recentemente inserido em uma Pilha (LIFO). O algoritmo explora um caminho até o fim antes de considerar alternativas. Com o conjunto de explorados, cada estado é expandido no máximo uma vez. A **Busca em Largura (BFS)**, por outro lado, **BFS** expande os nós usando uma Fila (FIFO). Ela busca todos os nós adjacentes ao atual antes de passar para o próximo e encontra o objetivo com o *menor número de travessias*. Como as travessias são ponderadas pelo tempo, um menor número de travessias não garante o menor tempo total.

3.3 Métodos de busca informada

A **Busca de Custo Uniforme (UCS)** é um método que ordena a fronteira baseando-se na informação sobre o custo g . O algoritmo expande sempre o nó de **menor custo acumulado** e **garante a solução ótima** em grafos com custos positivos. O método **A*** acelera a busca pelo caminho de menor custo ao usar alguma **heurística** $h(s)$ admissível para o problema. O algoritmo ordena a fronteira por $f(s) = g(s) + h(s)$, e, para busca em grafo com conjunto fechado e consistente consistente, retorna a **solução ótima expandindo menos nós**.

4 Heurísticas

Para análise da Busca Informada, foram considerados 3 diferentes heurísticas para o problema proposto

4.1 Heurística admissível h_1

$$h_1(s) = \max\{c_i : b_i = 0\} = \text{tempo da pessoa mais lenta que ainda precisa cruzar.}$$

Justificativa de admissibilidade. Todos que ainda estão no início da ponte **devem** necessariamente atravessar a ponte pelo menos uma vez em qualquer solução, uma vez que as travessias de volta só acrescentam custo. Portanto, o custo restante real $\geq \max_i c_i = h_1$

4.2 Heurística admissível mais forte h_2

Dado $c_{(1)} \geq c_{(2)} \geq \dots \geq c_{(k)}$ os **custos ordenados** de travessia das k pessoas restantes.

$$h_2(s) = c_{(1)} + c_{(3)} + c_{(5)} + \dots = \text{soma dos máximos de cada par } (c_{(2j-1)}, c_{(2j)}).$$

Justificativa. Se considerarmos o problema relaxado em que a tocha retorna sem custo e sempre está no lado necessário para travessia de uma pessoa, cada pessoa restante precisa de exatamente uma travessia de ida. Como e as idas particionam as k pessoas em grupos de até 2 com custo da pessoa mais lenta, o particionamento mínimo desse somatório combina os dois mais lentos entre si, depois os dois seguintes, etc., valendo exatamente h_2 . Se toda solução real *inclui* essas travessias de ida e *acrescenta* as voltas, $h_1(s) \leq h_2(s) \leq \text{custo restante real}$. h_2 é, portanto, admissível e estritamente mais informativa que h_1 sempre que houver pelo menos duas pessoas restantes.

4.3 Heurística inadmissível h_3

Soma de todos os tempos restantes:

$$h_3(s) = \sum_{i: b_i=0} c_i,$$

Ignora que pessoas viajam juntas e superestima o tempo total para o objetivo. Exemplo no caminho ótimo do problema proposto: Após 1,2 cruzarem e 1 voltar, restam $\{5,10\}$ do lado inicial, mas 1 e 2 estão disponíveis do outro lado; o custo real restante é $10 + 2 + 2 = 14$, enquanto $h_3 = 15 > 14$. h_3 foi incluída de para estudar o trade-off entre a força de guia e a perda de garantia de otimalidade.

5 Experimentação e Medições

5.1 Metodologia experimental

As metodologias usadas visaram a coleta de métricas limpas que não alterassem o comportamento medido. A instrumentação seguiu três princípios:

1. **Apenas um incremento no caminho crítico.** Os algoritmos atualizam um `struct SearchMetrics` com campos inteiros `expanded`, `generated`, `duplicates_skipped`, `max_frontier`, `max_explored`, `max_nodes` e `limit_hit`. Não há mutex, alocação, syscall nem E/S no laço de busca e o custo por atualização é da ordem de nanosegundos.
2. **Cronometragem externa à rotina.** O tempo é medido com `steady_clock` envolvendo exclusivamente a chamada da busca. A formatação, escrita de CSV e qualquer outra E/S ocorrem fora do trecho cronometrado.
3. **Custo da heurística isolado.** As heurísticas operam em um buffer de pilha (`std::array<int,31>`), sem alocação dinâmica, evitando a inflação artificial do custo por nó de A^* .

5.2 Rotinas de execução

- **Repetições:** 2 execuções de aquecimento seguida de 30 execuções cronometradas por configuração, reportando a *mediana* e o intervalo interquartil. Como os algoritmos são determinísticos, custo, expansões e demais contadores são **idênticos em todas as execuções**.
- **Instâncias:** Foram utilizadas a do enunciado ($n=4$, $\{1, 2, 5, 10\}$) e um benchmark escalável com $n \in \{4, 6, 8, 10, 12\}$ e dois vetores distintos de custos: custo aleatórios com sementes fixas e custo geométrico $\{1, 2, 4, \dots, 2^{n-1}\}$.
- **Referência de otimalidade:** o custo ótimo de cada instância é obtido por UCS com teto 10^8 e *independente* dos métodos em avaliação.
- **Verificação independente das soluções:** cada caminho retornado é revalidado percorrendo pares de estados consecutivos via XOR das máscaras para garantir que os calculos coincidem com o informado pelo algoritmo. Todos os caminhos passaram.
- **Volume:** 6 510 execuções registradas, uma linha de CSV por execução.

5.3 Pipeline de dados e geração dos gráficos

O binário grava um CSV por execução com 21 colunas (método, heurística, família, n , semente, tentativa, cap, achou, custo, custo ótimo, comprimento do caminho, recheck, expansões, gerações, duplicatas descartadas, pico de fronteira, pico de explorados, pico de nós, limite atingido, tempo em ns). Em seguida um script valida os dados gerados, agrega por configuração (mediana entre instâncias para cada n) e desenha cada figura em escala logarítmica, uma por método.

As Figuras 1–6 foram produzidas por esse pipeline: medianas de expansões (Fig. 1), tempo (Fig. 3), dispersão tempo $\times$ expansões com regressão linear por método (Fig. 4), desvio do ótimo (Fig. 2), pico de fronteira (Fig. 7) e o estudo de heurísticas no agregado e na família geométrica (Figs. 5 e 6).

5.4 Ambiente

Apple Silicon (ARM64), macOS, AppleClang, C++17, execução monothread; análise em Python 3.14 (pandas, matplotlib).

6 Resultados e discussão

6.1 Instância do enunciado {1,2,5,10}

Tabela 1: Médias de 30 execuções e tempo mediano com quartis. Ótimo = 17 minutos. mínimo de travessias = 5.

Método	Custo	Δ ótimo	Expandidos	Gerados	Fronteira	Ótimo?	Tempo (μ s)
BFS	19	+2	26	98	19	não	7,52 [7,39–7,62]
DFS	50	+33	7	29	19	não	3,88 [3,71–3,96]
UCS	17	0	26	98	21	sim	7,65 [7,34–7,98]
A* (h_0)	17	0	26	98	21	sim	7,58 [7,39–7,86]
A* (h_1)	17	0	19	71	24	sim	6,08 [6,04–6,51]
A* (h_2)	17	0	19	71	24	sim	6,21 [6,05–6,29]
A* (h_3)	17	0	17	59	27	sim	5,71 [5,59–5,90]

Análise da Tabela 1:

1. **BFS** achou 19 min: o primeiro encontro com o objetivo em largura não é o encontro mais barato. É exatamente por isso que BFS não garante otimalidade de *custo*. A garantia vale apenas quando todos os passos custam o mesmo.
2. **DFS** é o extremo oposto: menor esforço da tabela (7 expansões, 29 gerações, fronteira de pico 19) e menor tempo (3,88 μ s), porém a pior solução (50 min, +33). Aceitou a primeira rota completa que encontrou, sem avaliar alternativas mais baratas.
3. **UCS e A*** devolvem 17. O controle A* (h_0) reproduziu UCS exatamente. O guia h_1 cortou as expansões de 26 para 19 (−27%) e as gerações de 98 para 71, e o tempo caiu de 7,65 para 6,08 μ s. Menos nós compensam a tarifa extra por nó.
4. h_3 , **inadmissível**, achou o ótimo nesta instância com ainda menos esforço (17 expansões, 59 gerações), ao custo da fronteira mais alta (27). Aqui a heurística funcionou, mas a garantia, porém, pertence só a h_1 e h_2 .

6.2 Esforço de busca

Tabela 2: Mediana do nós expandidos usando custos aleatórios. n é o número de pessoas e o espaço tem 2^{n+1} estados.

Método	$n=4$	$n=6$	$n=8$	$n=10$	$n=12$
BFS	26	120	502	2036	8178
DFS	7	11	15	19	23
UCS	26	120	502	2034	8169
A* (h_0)	26	120	502	2034	8169
A* (h_1)	23	108	474	1952	8008
A* (h_2)	22	104	449	1867	7658
A* (h_3)	20	81	321	307	425

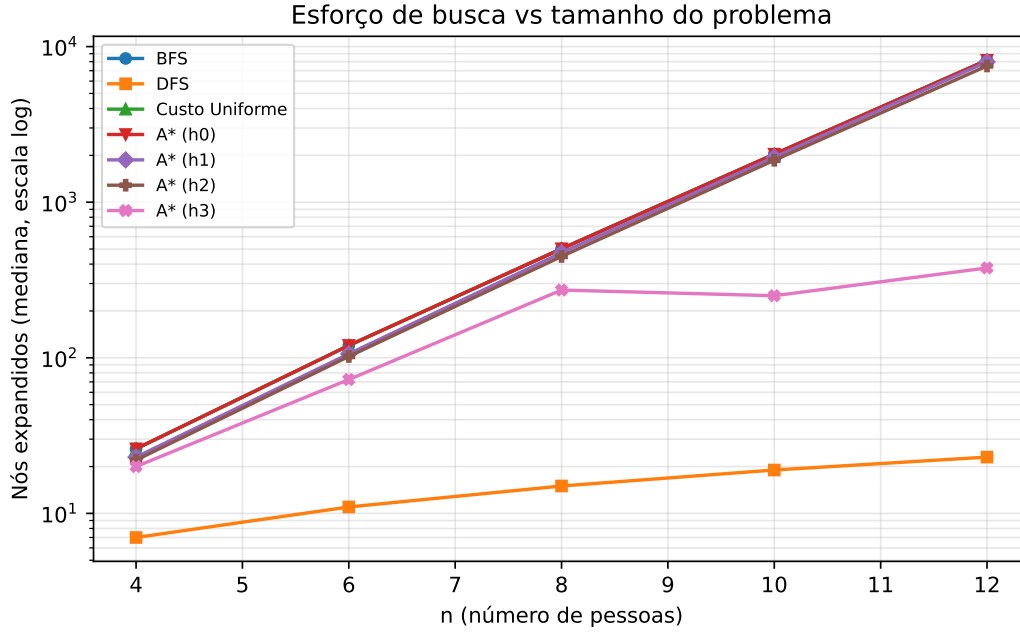


Figura 1: Mediana de nós expandidos, em escala log, conforme o número de pessoas cresce. Agregado das duas famílias de custos (aleatório e geométrico).

Análise: Em custos aleatórios (Tabela 2), UCS e $A^*(h_0)$ expandem 8169 dos 8192 estados existentes em $n=12$ (99,7%): sem informação, a busca de custo é basicamente uma *busca por exaustão*. BFS se comporta da mesma forma. A heurística h_1 economiza só $\sim 2\%$ e h_2 $\sim 6\%$. O valor da informação depende da estrutura de custos, como a Seção 6.5 mostra. Já h_3 , guiando pela soma, colapsa o esforço em até 95% (425 nós). DFS expande apenas 23 nós em $n=12$ (crescimento aproximadamente linear em n : 7, 11, 15, 19, 23), com a lista de explorados e a imersão LIFO, ele caminha quase reto até *uma* solução. (Seção 6.3).

6.3 Qualidade da solução

Tabela 3: Desvio da solução ótima em minutos (mediana por n). UCS e $A^*(h_0, h_1, h_2, h_3)$ tiveram desvio 0 em todas os testes.

Método	Custos aleatórios					Custos geométricos				
	4	6	8	10	12	4	6	8	10	12
BFS	0	5	10	25	43	1	14	75	328	1349
DFS	20	67	137	253	365	25	236	1479	8002	40253

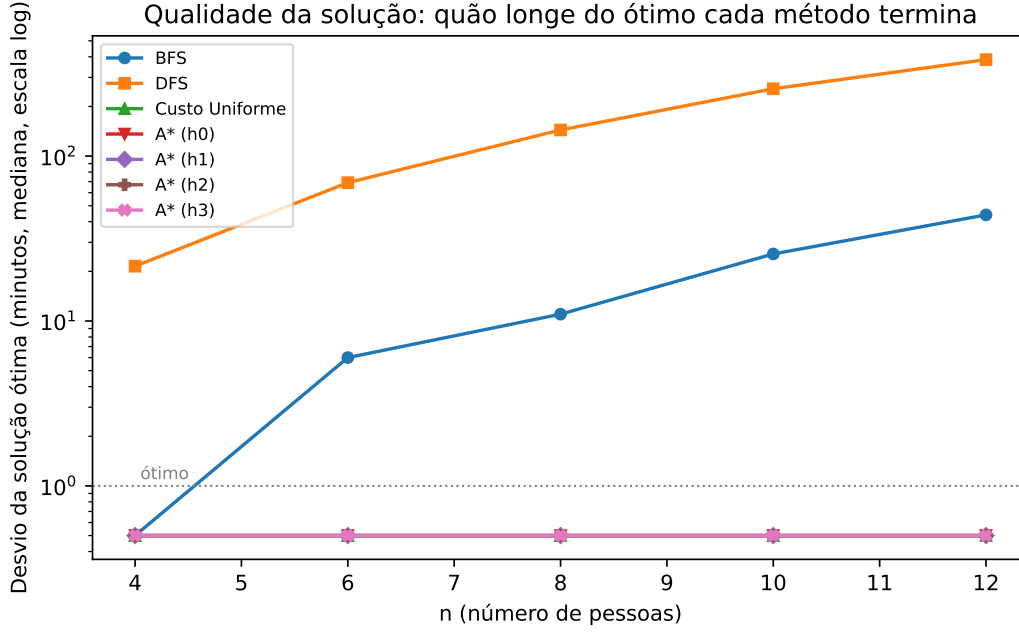


Figura 2: Desvio do custo ótimo (mediana, escala log).

Análise: A Tabela 3 e a Figura 2 quantificam a perda de garantia. BFS foi subótimo em 26 de 30 instâncias, com desvio mediano crescendo de 0 até 43 minutos (aleatórios) e até 1349 (geométricos). DFS foi subótimo em todas as 31 instâncias, atingindo desvio mediano de 40,253 minutos na geométrica $n=12$ — $14,6\times$ o ótimo (2755). Em $n=12$ geométrico, BFS entrega 4104 min (+49%). Nada na BFS/DFS pondera custos, a heurística de “menos travessias” ou de “profundidade” ignora completamente a economia de combinar os mais lentos e o desvio cresce exponencialmente com n porque o número de escolhas erradas cresce junto.

6.4 Tempo de processamento

Tabela 4: Custo marginal por nó: inclinação da regressão linear tempo $\times$ expansões (Figura 4), em ns por nó. É o custo que cada estratégia paga por nó expandido.

Método	ns/nó
DFS	729,5
BFS	990,3
UCS	1217,1
A* (h_0)	1276,8
A* (h_3)	1193,5
A* (h_1)	1552,8
A* (h_2)	1652,5

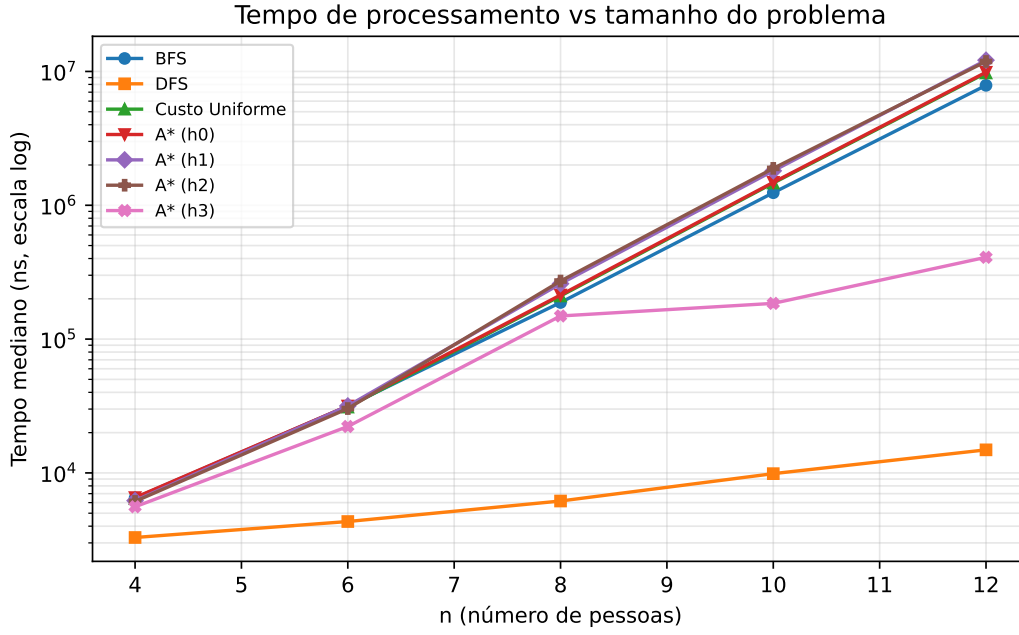


Figura 3: Tempo mediano de processamento por execução (escala log).

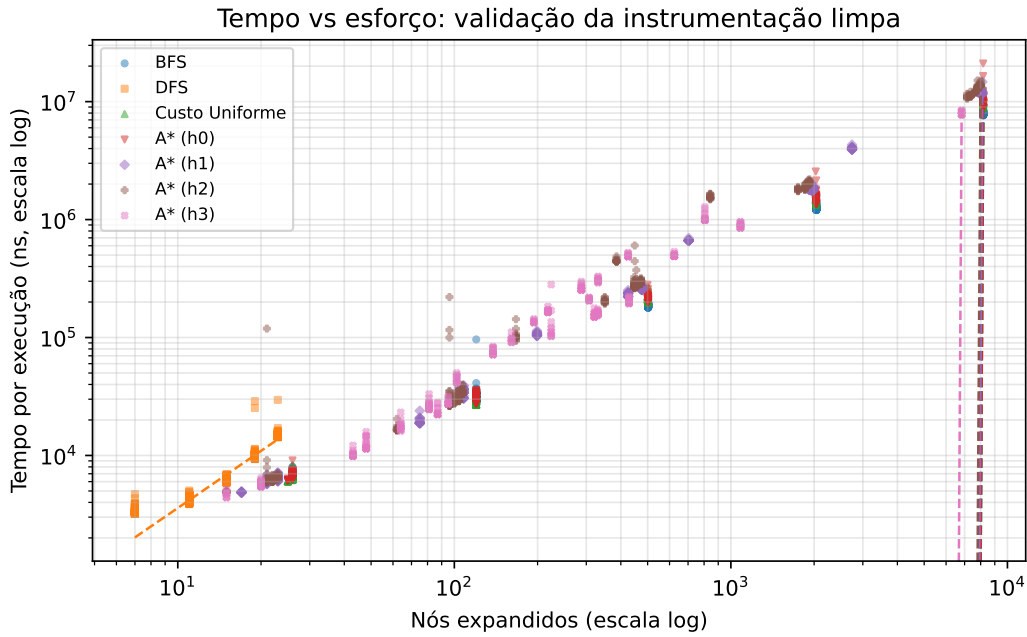


Figura 4: Tempo por execução vs nós expandidos.

Análise: A Figura 4 comprova a qualidade da instrumentação. O tempo de processamento é aproximadamente linear à quantidade de nós expandidos *dentro* de cada método, com coeficientes estáveis (Tabela 4). O Custo do processamento cresce com a complexidade da política de fronteira, como esperado ($DFS < BFS < UCS < A^*$) e, para as diferentes heurísticas, o custo ficou de acordo com a complexidade real de cada abordagem: h_3 (soma) quase não custa, h_1 (máximo) custa pouco, h_2 (ordenação em pilha) custa mais.

Nota-se também, que em $n=12$ geométrico, UCS leva 9,69 ms, enquanto $A^*(h_2)$ leva 1,56 ms ($6,2\times$ mais rápido) e $A^*(h_3)$ 0,17 ms ($57\times$). Nesse caso, a economia de nós ($9,7\times$ e $37\times$) paga

com sobra o custo mais alto por nó na busca informada. Já em $n=12$ aleatório, $A^*(h_1)$ expande 2% menos nós que UCS, mas é 25% *mais lento* (12,25 vs 9,76 ms). O custo por nó subiu 27% mas a economia foi de apenas 2%. Verifica-se, portanto, que *a heurística só compensa quando a informação economiza mais nós do que o custo extra por nó*. Na instância do problema proposto no enunciado, todos os tempos ficam entre 3,9 e 7,7 μ s com intervalos interquartis muito curtos (Tabela 1), e até $A^*(h_3)$ supera BFS em parede (5,71 vs 7,52 μ s).

6.5 Estudo da força da heurística

Tabela 5: Nós expandidos (custos geométricos).

Método	$n=4$	$n=6$	$n=8$	$n=10$	$n=12$	vs UCS
BFS	26	120	502	2036	8178	—
DFS	7	11	15	19	23	—
UCS	26	120	501	2034	8175	1,0×
$A^*(h_1)$	21	75	199	704	2742	3,0×
$A^*(h_2)$	21	62	167	386	842	9,7×
$A^*(h_3)$	20	43	81	138	218	37,5×

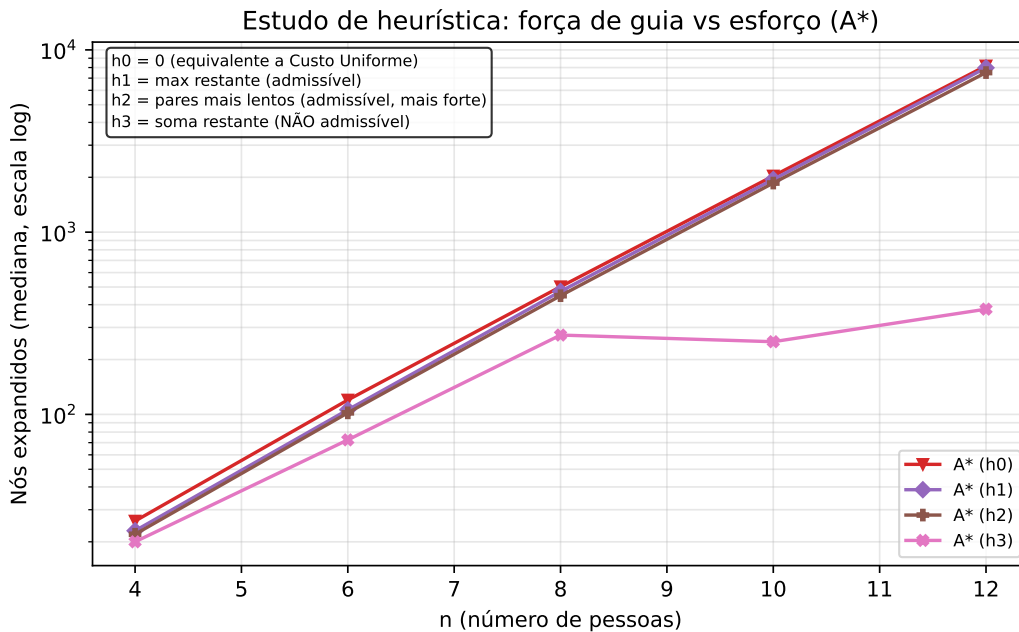


Figura 5: A^* sob quatro heurísticas: $h_0 \equiv 0$ reproduz UCS; admissíveis $h_1 \leq \text{real}$ e h_2 mais forte; h_3 inadmissível.

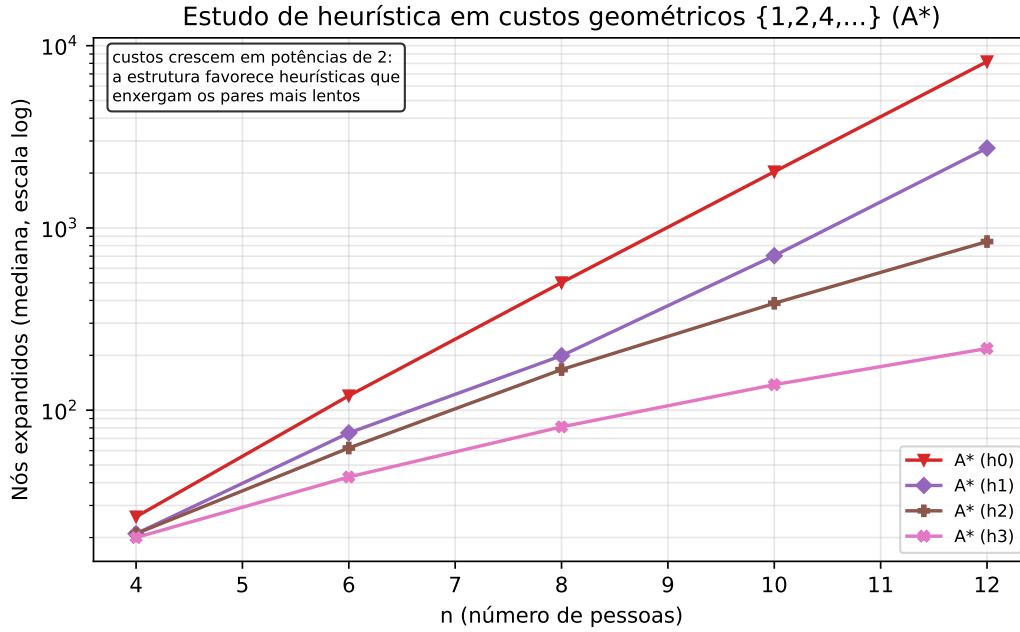


Figura 6: Estudo de heurística isolado nos custos geométricos.

A Figura 6 evidencia a hierarquia $h_0 > h_1 > h_2$ de esforço entre as heurísticas. Nos custos geométricos (Tabela 5, Fig. 6) a hierarquia de força vira curvas separadas: h_1 já economiza $3\times$, h_2 $9,7\times$ e h_3 $37,5\times$ em $n=12$. Como h_2 considera pares de pessoas restantes e o custo geométrico faz o custo dominado pelo par mais lento, a heurística aponta para a estratégia ótima de combinar os pares mais lentos.

Sobre h_3 : nos 30 benchmark ela *sempre* devolveu o ótimo. A checagem teórica (Seção 4) mostra estados do caminho ótimo em que h_3 superestima (15 vs 14), e em outras estruturas de custo ela pode de fato perder o ótimo. O experimento ilustra os dois lados do trade-off: h_3 foi o guia mais barato do benchmark, mas a garantia de otimalidade que h_1 e h_2 preservam é o que um agente “de produção” exige. Além disso, a curva de h_3 nos custos aleatórios não é monotônica ($321 \rightarrow 307 \rightarrow 425$ expansões de $n=8$ a 12). Ou seja, sem propriedades formais, nem o próprio comportamento é estável.

6.6 Memória

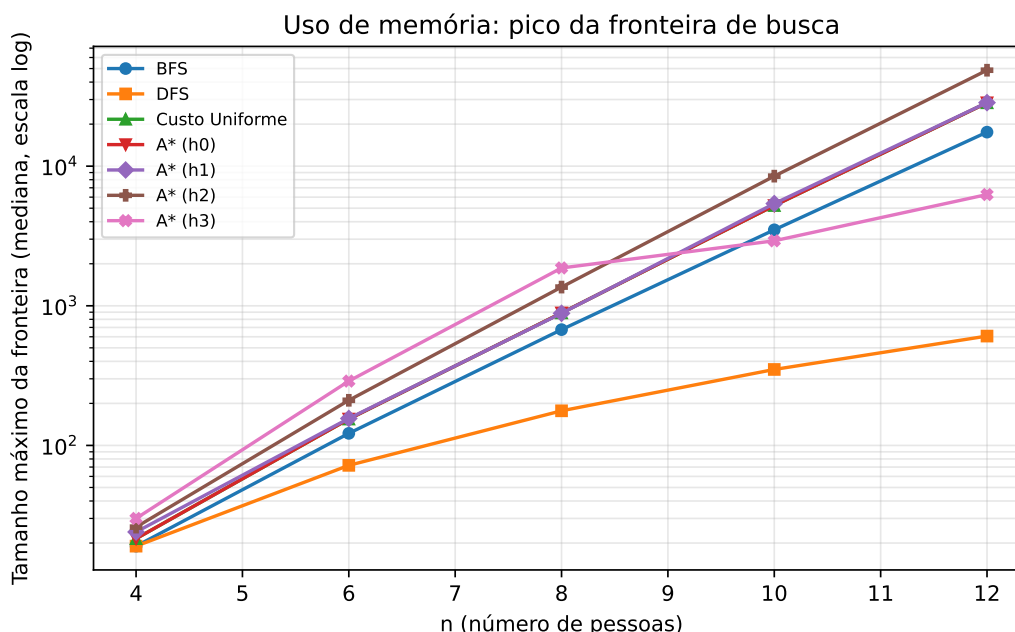


Figura 7: Pico de fronteira (proxy de memória de busca, mediana).

Em $n=12$ geométrico, o pico de fronteira foi de 26 393 nós para UCS/A*(h_0), 25 608 para A*(h_1), 15 205 para A*(h_2), 2 814 para A*(h_3) e apenas 607 para DFS (44× menor que UCS). Verifica-se que a fronteira de A*(h_2) é menor que a de UCS porque ele *expande* menos (o pico acompanha o esforço), mas em custos aleatórios a fronteira de A*(h_2) chega a *ultrapassar* a de UCS (36 993 vs 29 853 na mediana agregada), uma vez que A* guarda nós que nunca expande (guiar melhor não significa guardar menos). Além disso, DFS confirma o comportamento esperado: a fronteira cresce suavemente ($19 \rightarrow 607$) enquanto a de UCS explode exponencialmente.

7 Conclusão

Os experimentos realizados quantificaram a tese que sustenta as técnicas de busca em Inteligência Artificial: A informação sobre o problema, codificada como heurística, transforma a busca exaustiva em busca eficiente e direcionada. Foi possível verificar que UCS resolve corretamente o problema, mas esgota o espaço; A* com h_1/h_2 admissíveis devolve o mesmo ótimo com até 9,7× menos esforço e 6,2× menos tempo nos custos geométricos; heurísticas inadmissíveis sem garantia cobram seu preço de trade-off em confiança. Do lado não informado, DFS e BFS ilustram por que “encontrar uma solução” e “encontrar a melhor solução” são problemas diferentes: o custo das estratégias cegas aparece na qualidade (DFS) ou no esforço (BFS/UCS), nunca em ambos.

Referências

- RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. Pearson, 2020. (Cap. 3: Solução de problemas por meio da busca.)